

SISTEMA PATRIMONIAL

Documentação Técnica Completa

Spring Boot 3.2.3 | Java 17 | PostgreSQL 16 | Docker

Versão: 1.0.0

Artefato: sistema-patrimonial-1.0.0.jar

Data: Junho / 2026

Responsável: hussyvel

Sumário

1. Visão Geral do Sistema
2. Stack Tecnológica
3. Arquitetura
4. Modelo de Dados
    1. Entidade Bem
    2. Entidade Usuario
    3. Entidade Movimentação
5. API REST — Endpoints
    1. Autenticação (/api/auth)
    2. Bens (/api/bens)
    3. Movimentações (/api/movimentacoes)
    4. Usuários (/api/usuarios)
    5. Relatórios (/api/relatorios)
6. Segurança
7. Relatórios
8. Serviço de E-mail
9. Docker e Docker Compose
10. Referência de Comandos
11. Configuração do Ambiente
12. Resolução de Problemas

1. Visão Geral do Sistema

O **Sistema Patrimonial** é uma aplicação web para gerenciamento de patrimônio (bens materiais) de uma organização. Permite cadastrar bens, controlar empréstimos e devoluções, transferências entre setores, emitir relatórios e gerenciar usuários com diferentes perfis de acesso.

MÓDULO PRINCIPAL
Gestão de Bens (Patrimônio)

MÓDULO DE CONTROLE
Saídas, Devoluções e Transferências

MÓDULO DE RELATÓRIOS
PDF e Excel com filtros

MÓDULO DE USUÁRIOS
Admin e Operador (RBAC)

Funcionalidades Principais

FUNCIONALIDADE	DESCRIÇÃO	PERFIL
Cadastro de Bens	Criar, editar, excluir e buscar bens patrimoniais	Admin / Operador
Registro de Saída	Emprestar bem a um responsável com destino registrado	Admin / Operador
Registro de Devolução	Confirmar retorno do bem, restaura status para ATIVO	Admin / Operador
Transferência de Setor	Mover bem permanentemente entre setores	Admin / Operador
Dashboard	Totais por categoria, status e valor patrimonial	Admin / Operador
Relatório PDF	Relatório de bens com filtros, download direto	Admin
Relatório Excel	Planilha de bens com filtros, download direto	Admin
Gestão de Usuários	Criar, editar, ativar/inativar, excluir usuários	Admin
Recuperação de Senha	Envio de link de reset por e-mail	Público
Visão por Setor	Agrupar bens por setor com resumo	Admin / Operador

Credencial padrão: Na primeira execução, o sistema cria automaticamente um usuário `login: admin` / `senha: admin123` se não houver usuários cadastrados.

2. Stack Tecnológica

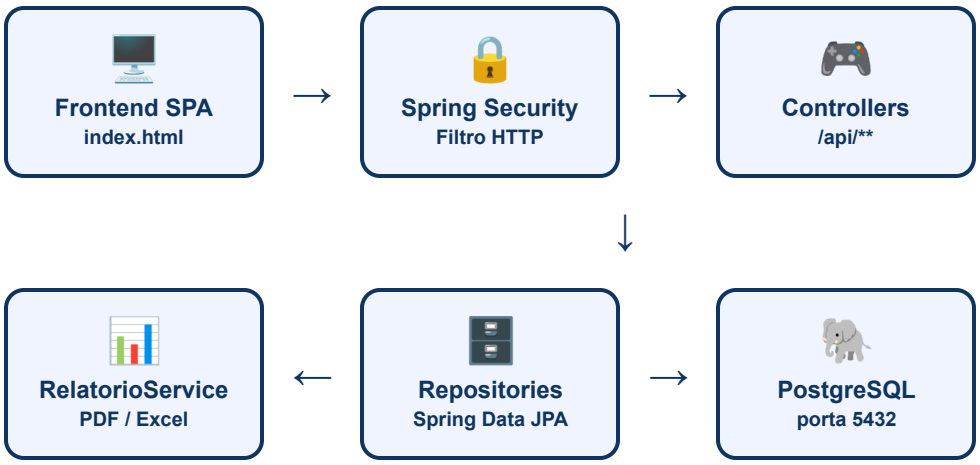
CAMADA	TECNOLOGIA	VERSÃO	FUNÇÃO
Linguagem	Java	17 (LTS)	Runtime da aplicação
Framework	Spring Boot	3.2.3	Auto-configuração, servidor embutido
Web	Spring MVC (REST)	6.1.x	Controllers REST, serialização JSON
Persistência	Spring Data JPA + Hibernate	3.2.x	ORM, JPQL, mapeamento de entidades
Banco de Dados	PostgreSQL	16	Banco principal
Segurança	Spring Security	6.2.x	Autenticação, autorização por perfil
Validação	Jakarta Validation (Bean Validation)	3.0	Validação de campos nas entidades
Relatório PDF	OpenPDF (fork do iText 2.1)	1.3.30	Geração de relatórios em PDF
Relatório Excel	Apache POI XSSF	5.x	Geração de planilhas .xlsx
E-mail	Spring Mail + Jakarta Mail	—	Envio de e-mail SMTP
Build	Maven	3.9.x	Gerenciamento de dependências e build
Container	Docker + Docker Compose	—	Empacotamento e orquestração
Frontend	HTML/CSS/JavaScript (SPA estático)	—	Interface servida pelo Spring Boot

Arquivo de Build — pom.xml (resumo)

```
<!-- Identificação do artefato --> groupId: br.gov.patrimonio artifactId: sistema-patrimonial version: 1.0.0 java: 17 <!-- Dependências principais --> spring-boot-starter-web spring-boot-starter-data-jpa spring-boot-starter-security spring-boot-starter-validation spring-boot-starter-mail postgresql (runtime) com.github.librepdf:openpdf org.apache.poi:poi-ooxml
```

3. Arquitetura

Arquitetura em camadas padrão Spring Boot. **Não há camada de Service** exceto para geração de relatórios e envio de e-mail — os controllers chamam os repositories diretamente.



Estrutura de Pacotes

```
src/main/java/patrimonio/ ├── PatrimonioApplication.java # Classe main (Spring Boot) ├── config/ │ ├── SecurityConfig.java # Regras de segurança HTTP │ └── DataInitializer.java # Seed do usuário admin ├── model/ │ ├── Bem.java # Entidade principal (tabela: bens) │ ├── Usuario.java # Usuários do sistema (tabela: usuarios) │ ├── Movimentacao.java # Saídas/devoluções (tabela: movimentacoes) │ └── Setor.java # Enum de setores disponíveis ├── repository/ │ ├── BemRepository.java # Queries JPQL de Bem │ ├── UsuarioRepository.java # Queries de Usuario │ └── MovimentacaoRepository.java # Queries de Movimentacao ├── controller/ │ ├── AuthController.java # Login, logout, /me │ ├── BemController.java # CRUD de bens + dashboard │ ├── UsuarioController.java # CRUD de usuários │ ├── MovimentacaoController.java # Saídas, devoluções, transferências │ ├── RelatorioController.java # Geração de PDF/Excel │ └── PasswordResetController.java # Reset de senha por e-mail └── service/ ├── RelatorioService.java # Lógica de geração PDF + Excel └── EmailService.java # Envio de e-mail SMTP src/main/resources/ ├── application.properties # Configurações da aplicação └── static/ └── index.html # Frontend SPA (Single Page Application)
```

4. Modelo de Dados

4.1 Entidade Bem (tabela: bens)

CAMPO	TIPO JAVA	COLUNA DB	OBRIGATÓRIO	DESCRIÇÃO
id	Long	id (PK, SERIAL)	—	Identificador gerado automaticamente
numeroPatrimonio	String	numero_patrimonio	✓ (único)	Número de tombamento
descricao	String	descricao	✓	Descrição do bem
categoria	Categoria (enum)	categoria	✓	NOTEBOOK, DESKTOP, DRONE, MONITOR, IMPRESSORA, PERIFERICO, OUTRO
marca	String	marca	—	Fabricante
modelo	String	modelo	—	Modelo do equipamento
numeroSerie	String	numero_serie	—	Número de série
status	Status (enum)	status	—	ATIVO (padrão), MANUTENCAO, EMPRESTADO, BAIXADO
localizacao	String	localizacao	—	Localização física do bem
setor	String	setor	—	Setor responsável
responsavel	String	responsavel	—	Nome do responsável
responsavelCpf	String	responsavel_cpf	—	CPF do responsável
responsavelEmail	String	responsavel_email	—	E-mail do responsável
responsavelTelefone	String	responsavel_telefone	—	Telefone do responsável
dataAquisicao	LocalDate	data_aquisicao	—	Data de compra
valor	BigDecimal	valor	—	Valor de aquisição (12,2)

hostname	String	hostname	—	Nome de rede (computadores)
observacoes	String	observacoes (TEXT)	—	Observações livres
criadoEm	LocalDateTime	criado_em	—	Timestamp de criação (imutável)
atualizadoEm	LocalDateTime	atualizado_em	—	Timestamp da última atualização

4.2 Entidade Usuario (tabela: usuarios)

CAMPO	TIPO JAVA	OBRIGATÓRIO	DESCRIÇÃO
id	Long	—	PK gerada automaticamente
nome	String	✓	Nome completo
login	String	✓ (único)	Login de acesso ao sistema
email	String	—	E-mail para recuperação de senha
senha	String	✓	Hash BCrypt da senha
perfil	Perfil (enum)	—	ADMIN ou OPERADOR (padrão: OPERADOR)
ativo	boolean	—	false = usuário bloqueado do login
criadoEm	LocalDateTime	—	Timestamp de criação

4.3 Entidade Movimentacao (tabela: movimentacoes)

CAMPO	TIPO JAVA	OBRIGATÓRIO	DESCRIÇÃO
id	Long	—	PK gerada automaticamente
bem	Bem (@ManyToOne)	✓	Bem vinculado (FK: bem_id)
tipo	Tipo (enum)	✓	SAIDA, DEVOLUCAO ou TRANSFERENCIA
responsavelRetirada	String	✓	Nome de quem levou o bem
cpfRetirada	String	—	CPF de quem retirou
localOrigem	String	—	Localização antes da saída
destino	String	—	Destino do bem
dataSaida	LocalDateTime	✓	Momento do registro de saída
dataDevolucao	LocalDateTime	—	null = em aberto; preenchido = devolvido

usuarioSaida	String	—	Login do funcionário que registrou a saída
usuarioDevolucao	String	—	Login do funcionário que registrou a devolução
numeroChamado	String	—	Número de chamado / OS
obsSaida	String	—	Observações da saída
obsDevolucao	String	—	Observações da devolução
setorAnterior	String	—	Setor antes da movimentação
setorAtual	String	—	Setor atual após movimentação

Regra de negócio: Uma movimentação é considerada *em aberto* quando `dataDevolucao == null`. Um bem com saída em aberto não pode receber nova saída nem ser transferido — é necessário registrar a devolução primeiro.

5. API REST — Endpoints

5.1 Autenticação — /api/auth

POST `/api/auth/login`
Realiza login. Cria sessão HTTP. Body: `{"login":"admin","senha":"admin123"}`
Público

GET `/api/auth/me`
Retorna dados do usuário autenticado na sessão atual
Autenticado

POST `/api/auth/logout`
Encerra a sessão do usuário atual
Autenticado

5.2 Bens — /api/bens

GET `/api/bens`
Lista todos os bens. Query param opcional: `?q=texto` para busca
Autenticado

GET `/api/bens/{id}`
Retorna um bem específico pelo ID
Autenticado

POST `/api/bens`
Cria novo bem. Body: objeto Bem (numeroPatrimonio, descricao e categoria obrigatórios)
Autenticado

PUT `/api/bens/{id}`
Atualiza bem existente. Setor bloqueado se houver saída em aberto
Autenticado

DELETE `/api/bens/{id}`
Exclui um bem e todas as suas movimentações
Autenticado

DELETE `/api/bens`
Exclui TODOS os bens e movimentações (operação irreversível)
Autenticado

GET **/api/bens/dashboard**
Retorna totais gerais, contagem por categoria e por status
Autenticado

GET **/api/bens/enums**
Retorna listas de categorias, statuses e setores disponíveis
Autenticado

GET **/api/bens/por-setor**
Retorna bens agrupados por setor com resumo de quantidades
Autenticado

5.3 Movimentações — /api/movimentacoes

GET **/api/movimentacoes**
Lista todas as movimentações (ordenadas por data desc). Suporta `?q=`
Autenticado

GET **/api/movimentacoes/em-aberto**
Lista apenas saídas sem devolução registrada
Autenticado

GET **/api/movimentacoes/bem/{bemId}**
Histórico completo de movimentações de um bem específico
Autenticado

POST **/api/movimentacoes/saida**
Registra saída de bem. Altera status do bem para EMPRESTADO. Body: `{bemId, responsavelRetirada, destino, ...}`
Autenticado

POST **/api/movimentacoes/devolucao/{movId}**
Registra devolução. Restaura status do bem para ATIVO e localização de origem
Autenticado

POST **/api/movimentacoes/transferencia**
Transfere bem entre setores permanentemente. Body: `{bemId, setorDestino, responsavel}`
Autenticado

GET **/api/movimentacoes/resumo**
Contagem de saídas em aberto e total de movimentações
Autenticado

5.4 Usuários — /api/usuarios

GET `/api/usuarios`
Lista todos os usuários (senhas omitidas na resposta)
Somente ADMIN

POST `/api/usuarios`
Cria novo usuário. Body: `{nome, login, email, senha, perfil}`. Senha armazenada com BCrypt
Somente ADMIN

PUT `/api/usuarios/{id}`
Atualiza usuário. Senha só alterada se informada no body
Somente ADMIN

DELETE `/api/usuarios/{id}`
Exclui usuário pelo ID
Somente ADMIN

5.5 Relatórios — /api/relatorios

GET `/api/relatorios/bens?formato=pdf`
Gera e baixa relatório PDF com filtros opcionais: `categoria`, `responsavel`, `cpf`, `dataInicio`, `dataFim`
Somente ADMIN

GET `/api/relatorios/bens?formato=excel`
Gera e baixa planilha Excel (.xlsx) com os mesmos filtros
Somente ADMIN

6. Segurança

Autenticação baseada em sessão HTTP gerenciada pelo Spring Security. CSRF desabilitado. Senhas armazenadas com **BCrypt**.

Regras de Acesso (SecurityConfig)

ROTA	MÉTODO	ACESSO
/api/auth/**	Todos	Público
/	GET	Público
/api/bens/**	GET	Autenticado
/api/movimentacoes/**	Todos	Autenticado
/api/bens/**	POST, PUT, DELETE	Autenticado
/api/usuarios/**	Todos	ROLE_ADMIN
/api/relatorios/**	Todos	ROLE_ADMIN

Perfis de Usuário

PERFIL	ROLE SPRING SECURITY	PERMISSÕES
ADMIN	ROLE_ADMIN	Acesso total: bens, movimentações, usuários, relatórios
OPERADOR	ROLE_OPERADOR	Consultar bens, registrar saídas e devoluções, transferências

Usuário inativo: Se `ativo=false`, o Spring Security lança `DisabledException` e o login retorna 401. O campo é gerenciado por ADMIN via PUT `/api/usuarios/{id}`.

7. Relatórios

Gerados pelo `RelatorioService` e disponibilizados via `RelatorioController` com download direto (Content-Disposition: attachment).

Filtros Disponíveis

PARÂMETRO	TIPO	EXEMPLO	DESCRIÇÃO
categoria	String	NOTEBOOK	Filtra por categoria do bem
responsavel	String	João Silva	Filtra por nome do responsável
cpf	String	123.456.789-00	Filtra por CPF do responsável
dataInicio	LocalDate (ISO)	2026-01-01	Data de aquisição mínima
dataFim	LocalDate (ISO)	2026-12-31	Data de aquisição máxima
formato	String	pdf ou excel	**Obrigatório** — define o formato de saída

Exemplos de URL

```
# Relatório completo em PDF GET /api/relatorios/bens?formato=pdf # Apenas notebooks em Excel GET /api/relatorios/bens?formato=excel&categoria=NOTEBOOK # Bens de um responsável específico GET /api/relatorios/bens?formato=pdf&responsavel=João+Silva # Bens adquiridos em 2025 GET /api/relatorios/bens?formato=excel&dataInicio=2025-01-01&dataFim=2025-12-31
```

Nomenclatura dos arquivos: Os arquivos baixados têm timestamp automático no nome, ex.: `relatorio_bens_20260609_1430.pdf`

8. Serviço de E-mail

Utilizado para o fluxo de recuperação de senha. Configurado via SMTP no `application.properties`.

Configuração Gmail (ativa)

```
spring.mail.host=smtp.gmail.com spring.mail.port=587
spring.mail.username=hussyribeiro@gmail.com spring.mail.password=<app-password-gmail> spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
spring.mail.properties.mail.smtp.from=hussyribeiro@gmail.com
```

Fluxo de Recuperação de Senha

ENDPOINT	DESCRIÇÃO
POST /api/auth/esqueci-senha	Recebe o e-mail, gera token e envia link de reset
POST /api/auth/redefinir-senha	Recebe token + nova senha, atualiza BCrypt hash

> **Gmail:** Use uma *App Password* (senha de app) gerada em Conta Google → Segurança → Senhas de app. A senha normal de login não funciona com SMTP.

9. Docker e Docker Compose

O projeto inclui **Dockerfile** com build multi-estágio e **docker-compose.yml** que orquestra a aplicação + PostgreSQL. Resolve o problema de versão de Java e conflito de portas.

Dockerfile (multi-estágio)

```
# Estágio 1: build com Maven + Java 17 FROM maven:3.9.6-eclipse-temurin-17 AS build WORKDIR /app COPY pom.xml . RUN mvn dependency:go-offline -q COPY src ./src RUN mvn package -DskipTests -q # Estágio 2: imagem final enxuta FROM eclipse-temurin:17-jre-jammy WORKDIR /app COPY --from=build /app/target/*.jar app.jar EXPOSE 8080 ENTRYPOINT ["java", "-jar", "app.jar"]
```

docker-compose.yml

```
services: db: image: postgres:16-alpine container_name: patrimonio-db environment: POSTGRES_DB: patrimonio POSTGRES_USER: postgres POSTGRES_PASSWORD: postgres ports: - "5433:5432" # 5433 no host (não conflita com Postgres local) volumes: - patrimonio-db-data:/var/lib/postgresql/data healthcheck: test: ["CMD-SHELL", "pg_isready -U postgres -d patrimonio"] interval: 5s retries: 10 app: build: . container_name: patrimonio-app ports: - "8082:8080" # acessa em http://localhost:8082 environment: SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/patrimonio SPRING_DATASOURCE_USERNAME: postgres SPRING_DATASOURCE_PASSWORD: postgres depends_on: db: condition: service_healthy volumes: patrimonio-db-data:
```

URL DA APLICAÇÃO (DOCKER)
http://localhost:8082

POSTGRESQL NO HOST (DOCKER)
localhost:5433

10. Referência de Comandos

Build e Execução (sem Docker)

```
# Compilar o projeto JAVA_HOME=/usr/lib/jvm/java-1.17.0-openjdk-amd64 mvn compile # Build completo (gera o JAR em target/) JAVA_HOME=/usr/lib/jvm/java-1.17.0-openjdk-amd64 mvn clean install # Subir em modo desenvolvimento (hot-reload não habilitado) JAVA_HOME=/usr/lib/jvm/java-1.17.0-openjdk-amd64 mvn spring-boot:run # Subir em porta diferente JAVA_HOME=/usr/lib/jvm/java-1.17.0-openjdk-amd64 mvn spring-boot:run \ -Dspring-boot.run.arguments="--server.port=8082" # Executar o JAR gerado diretamente JAVA_HOME=/usr/lib/jvm/java-1.17.0-openjdk-amd64 \ java -jar target/sistema-patrimonial-1.0.0.jar
```

Docker Compose — Comandos Essenciais

```
# Primeira vez ou após mudanças no código Java docker compose up --build # Subir sem recompilar (código não mudou) docker compose up # Subir em background (detached) docker compose up -d # Parar todos os containers docker compose down # Parar e apagar o volume do banco (dados perdidos!) docker compose down -v # Ver logs da aplicação em tempo real docker compose logs -f app # Ver logs do banco de dados docker compose logs -f db # Ver status dos containers docker compose ps # Entrar no container da aplicação (shell) docker compose exec app sh # Entrar no PostgreSQL dentro do Docker docker compose exec db psql -U postgres -d patrimonio
```

Maven — Comandos Adicionais

```
# Limpar diretório target/ mvn clean # Build sem rodar testes mvn package -DskipTests # Ver árvore de dependências mvn dependency:tree # Verificar atualizações de dependências mvn versions:display-dependency-updates
```

PostgreSQL — Acesso Direto (local)

```
# Conectar ao banco local psql -U postgres -d patrimonio # Principais consultas úteis SELECT * FROM bens LIMIT 10; SELECT * FROM usuarios; SELECT * FROM movimentacoes WHERE data_devolucao IS NULL; # Contar bens por status SELECT status, COUNT(*) FROM bens GROUP BY status; # Bens emprestados no momento SELECT b.numero_patrimonio, b.descricao, m.responsavel_retirada,
```

```
m.data_saida FROM movimentacoes m JOIN bens b ON b.id = m.bem_id WHERE
m.data_devolucao IS NULL;
```

Configuração Permanente do Java 17 (opcional)

```
# Adicionar ao ~/.bashrc para não precisar do JAVA_HOME em todo comando
echo 'export JAVA_HOME=/usr/lib/jvm/java-1.17.0-openjdk-amd64' >> ~/.bashrc
echo 'export PATH=$JAVA_HOME/bin:$PATH' >> ~/.bashrc source ~/.bashrc #
Verificar java -version # deve mostrar openjdk 17
```

11. Configuração do Ambiente

Pré-requisitos

COMPONENTE	VERSÃO MÍNIMA	VERIFICAÇÃO
Java JDK	17	`java -version`
Maven	3.6+	`mvn --version`
PostgreSQL	12+	`psql --version`
Docker	24+	`docker --version`
Docker Compose	2.x	`docker compose version`

application.properties — Configurações Chave

PROPRIEDADE	VALOR PADRÃO	DESCRIÇÃO
server.port	8080	Porta HTTP da aplicação
spring.datasource.url	jdbc:postgresql://localhost:5432/patrimonio	URL de conexão ao banco
spring.datasource.username	postgres	Usuário do PostgreSQL
spring.datasource.password	postgres	Senha do PostgreSQL
spring.jpa.hibernate.ddl-auto	update	Schema gerenciado automaticamente pelo Hibernate
spring.jpa.show-sql	false	Logar queries SQL no console
spring.mail.host	smtp.gmail.com	Servidor SMTP para e-mails
spring.mail.port	587	Porta SMTP (STARTTLS)

Substituição por Variável de Ambiente (Docker)

No `docker-compose.yml`, as propriedades de `application.properties` podem ser sobrescritas via variável de ambiente usando a notação do Spring:

```
# application.properties spring.datasource.url =
jdbc:postgresql://localhost:5432/patrimonio # Equivalente como variável de
```

```
ambiente (Docker Compose) SPRING_DATASOURCE_URL:
jdbc:postgresql://db:5432/patrimonio
```

12. Resolução de Problemas

PROBLEMA	CAUSA	SOLUÇÃO
UnsupportedClassVersionError (class file version 61.0)	Maven rodando com Java 11 em vez de Java 17	Prefixar o comando com `JAVA_HOME=/usr/lib/jvm/java-1.17.0-openjdk-amd64` ou configurar no `~/.bashrc`
Port 8080 already in use	Outro processo usando a porta	Identificar com `sudo lsof -i:8080` e matar, ou usar porta diferente com `--server.port=8082`
Connection refused (PostgreSQL)	PostgreSQL não está rodando	Iniciar com `sudo systemctl start postgresql` ou usar `docker compose up db`
FATAL: database "patrimonio" does not exist	Banco não criado	Criar com `createdb -U postgres patrimonio` ou deixar o Docker criar automaticamente
401 em todas as requisições	Sessão expirada ou não autenticado	Fazer login novamente em `POST /api/auth/login`
403 Forbidden	Usuário OPERADOR acessando rota de ADMIN	Usar conta com perfil ADMIN para /api/usuarios/** e /api/relatorios/**
Docker: image not rebuilding	Cache do Docker	Forçar rebuild com `docker compose build --no-cache`
E-mail de reset não chega	App Password do Gmail inválida ou SMTP bloqueado	Gerar nova App Password em Conta Google → Segurança → Senhas de app

Java 17 instalado mas não reconhecido pelo Maven?
Verifique quais versões estão disponíveis: `update-java-alternatives -l`
Use sempre o prefixo `JAVA_HOME=...` ou troque o Java padrão do sistema: `sudo update-alternatives --config java`

Sistema Patrimonial v1.0.0
Documentação gerada em Junho / 2026
Spring Boot 3.2.3 · Java 17 · PostgreSQL 16